



Federated learning-based intrusion detection system for Internet of Things

Najet Hamdi¹

Published online: 14 July 2023

© The Author(s), under exclusive licence to Springer-Verlag GmbH, DE 2023

Abstract

Intrusion detection in the Internet of Things is becoming increasingly important as the number of connected devices grows. Machine learning algorithms can be applied to detect anomalies in large data sets, making them useful for identifying potential intrusions. However, traditional centralized learning techniques entail collecting data from end devices in one central device for training. Allowing a single entity to have access to vast amounts of personal data raises many security concerns as any issue experienced with the system can lead to widespread data leakage. To prevent these issues, it is critical to seek more secure alternatives such as federated learning. It enables multiple parties to collaborate on the same model without having to share the data between them. This process not only helps protect data privacy, but also reduces the risk of data leakage and improves training efficiency. In this paper, we propose a federated-based intrusion detection system. To better investigate the performance of the proposed model, we considered client-side evaluation whereby in the same round, the clients transfer the local models to the server which aggregates them in an updated global model. Then, the server transfers the updated global model to the clients for evaluation. The clients evaluate the global model locally and send back the results to the server to be aggregated using metric aggregation function. The experimental results show that the proposed federated-IDS achieves a high detection rate.

Keywords Federated learning · Centralized learning · Internet of Things · Communication overhead · Cyberattacks

1 Introduction

The growing popularity of the Internet of Things (IoT) has resulted in a massive increase in connected devices, resulting in a highly complex and heterogeneous network. With the increased connectivity IoT, networks security concerns have grown significantly. One way to address security challenges is implementing Intrusion Detection Systems (IDS).

Intrusion detection systems seek to identify harmful or anomalous network activity. To detect possible threats, these systems employ a variety of approaches, including signature-based detection and anomaly detection. signature-based detection uses signatures (also known as patterns) to compare network traffic to a database of known attack patterns. If a match is discovered, it will be marked as malicious and blocked. Anomaly detection involves training the system

with a baseline of normal activities to evaluate the current activity to identify any deviations from normal network behavior, which may indicate a malicious attack.

Machine learning algorithms are key component for creating effective anomaly detection systems. The learning process could be either centralized, distributed or federated. Centralized learning involves collecting data from end devices and transmitting it to a central server to create a centralized dataset to train a centralized model. Federated learning [16] is a collaborating learning by which end devices participate in training a global model using their local datasets. In distributed learning, training and datasets are split up and shared among multiple end devices.

The fundamental issue with centralized and distributed training is that huge data transfer from edge devices to central servers might cause network congestion. Furthermore, an attacker can easily get access to the centralized server. If the obtained dataset contains sensitive information, there might be a serious security problem. On the other hand, federated learning enables collaborative training while keeping datasets on devices, ensuring data privacy. An aggregator

✉ Najet Hamdi
elhamdi.najet@gmail.com

¹ Department of Computer Science, Higher Institute of Technological Studies ISET, Medenine, Tunisia

server sends global model parameters to cooperating end devices (referred to as clients). Each client trains the model on its own dataset and returns the model parameters. The server aggregates all of the collected parameters and updates the global model, which is then transmitted back to the clients for more training. The process is repeated until the desired results are achieved. Therefore, federated learning is a pertinent solution to create IDSs.

Many federated-based IDS strategies were proposed [2, 7, 21, 27]. However, despite these efforts, multiple limitations remain:

- In fact, most of the proposed studies partially examined federated performance: only server-side evaluation was looked upon. Even though the model is trained locally, the evaluation is conducted at the server side which does not reflect the real performance as each client has its local dataset which can change over time. Besides, it is important to assess per-class evaluation for each client as high accuracy rates can result from a focus on majority classes that often leads to ignoring minority classes.
- Most of the proposed methods did not investigate the convergence of federate-based models. In fact, the communication overhead of federated learning is becoming difficult owing to constant cycles of model parameter download and upload. Decreasing the number of communication rounds without losing model performance is one technique to decrease communication overhead. Considering the convergence of federate-based models helps in finding a balance between minimizing communication rounds and achieving good performance.

Taking into account the above limitations, we propose, in this study, a federate-based intrusion detection model for IoT environments. We used the CSE-CIC-IDS2018 dataset [26] to train the proposed model. The main contributions of this paper are summarized below:

- We propose a federated-based model for intrusion detection. The proposed model is compared to its centralized and individual counterparts.
- We adopt a per-client evaluation, the different performance metrics are evaluated for each client and for each class. The convergence and the communication efficiency of the federated model is also investigated.
- We conduct experiments using a real-world dataset (CSE-CIC-IDS2018 dataset [26]) to evaluate the effectiveness of the proposed intrusion detection model.

This paper is organized as follows: Sect. 2 presents related works on federated-based IDS for IoT. Section 3 details the proposed federated-based IDS. The experimental results

are presented and discussed in Sect. 4, followed by future research directions in Sect. 5. Section 6 concludes with closing remarks and future directions.

2 Related work

In this section, we highlight the contributions and limitations of existing federated-based IDS methods.

Talbini et al. [27] proposed a three-tier clustering architecture for federated-based IDS. The first tier consists of end devices, the middle tier consists of data aggregators acting as federated clients, while the third tier is the central server. Data aggregator nodes collect data from subsets of clustered end devices. However, the proposed strategy flouts the data privacy feature of federated learning as data are transferred from end devices to aggregator nodes.

Ferrag et al. [7] evaluated federated learning with three deep learning models: convolutional neural network (CNN), recurrent neural network (RNN) and deep neural network (DNN). Three datasets were used for each model. The suggested research did not look into individual client performance; only the worst and best accuracies were given. It would be interesting to investigate if the high accuracy rate is due to neglecting minority groups. Furthermore, the authors provided non-IID datasets experiments but did not clarify whether local datasets have distinct features, distinct classes, or just different class distributions.

Rashid et al. [21] investigated federated-based IDS for IoT. The authors deployed two federated models: CNN and RNN. Similar to [7], the study did not investigate per-client performance such as per-class recall, precision or F1-score.

Li et al. [13] suggested a federated deep learning framework consisting of 3 modules: CNN, gated recurrent unit (GRU) and a multilayer perceptron (MLP). The outputs of multi-classification CNN-GRU model are concatenated and fed to the MLP module. However, the convergence of the proposed platform is not discussed.

The efficacy of federated learning in detecting network threats was evaluated by Cetin et al. [3]. The learning procedure involved 1000 customers. However, the research provided only the accuracy metric rate.

Chen et al. [4] presented a federated learning-based attention gated recurrent unit (FedAGRU) algorithm for intrusion detection. The model parameters are weighted for each client. The contribution of the client to the aggregated model detection is proportional to its weight. If the received local model's weight in the current round is less than a minimum threshold, it is not considered in the aggregation step at the server. Clients with local models weighing less than the minimum threshold for certain consecutive rounds are permanently excluded from participating in the learning process. They only receive the updated model. Excluding some clients

from the aggregation process could result in adverse impact especially in case of non-IID data as excluded datasets may include unique types of attacks. Besides, the authors did not evaluate the performance of dropped clients (temporarily or permanently dropped).

Arya et al. [2] proposed intrusion detection for vehicular adhoc networks (VANETs) based on federated learning. To aggregate the different local models, the proposed approach apply weighting mechanism for the collaborating clients (smart vehicles). However, similar to [4], then per-client performance is not investigated which is critical for assessing the effect of the weighting approach.

Huong et al. [17] used federated learning to detect network attacks. A neural network (NN) multi-class classifier was proposed and evaluated in centralized and federated schemes. As observed by [7, 21], The centralized model outperformed its federated counterpart.

Table 1 outlines the main features of these studies. We have taken into account a number of factors to categorize these works, including: number of attacks, training datasets, ML algorithms, number of collaborating clients, evaluation strategy (server-side or client evaluation), and model's convergence.

Though federated-based IDS were investigated by multiple approaches, many limitations and requirements continue to exist. Among these limitations, we cite:

- Client performance evaluation was lacking in most of the suggested methods, which tended to use centralized evaluation (or server-side evaluation). It operates in a similar manner to centralized learning evaluation. Utilizing a server-side dataset, the global model is evaluated at the server. In contrast to centralized evaluation, federated evaluation is more representative. It is conducted at the client-side: upon the reception of the global model, the client evaluates it using its local dataset. Few studies [7, 17, 21] provided only per-client accuracy and loss. Despite the fact that the majority of the suggested IDS models are multi-class classifiers, per-class performance was not examined.
- Convergence of federated model: Convergence occurs when a model's loss settles around its minimum over time, indicating that more training will not enhance the model's performance. The model's convergence is critical since it establishes the balance between performance and communication overhead. In fact, training for high number of rounds has a higher communication overhead, whereas a low number of rounds may compromise the model's performance. Despite this, the majority of the suggested techniques did not investigate the convergence of their proposed models.

3 Proposed method

The suggested strategy seeks to create a reliable detection system. We created a multi-class classifier using a federated-based conventional neural network (CNN). We used imbalanced datasets with distinct class distributions for each device.

3.1 Federated learning process

The learning process adopted in our federated-based CNN model is as follows:

1. At the start up the server gets the initial model parameters form one of the connected clients, randomly chosen. The initial parameters are considered as initial global model parameters.
2. The server sends the global model parameters to all connected clients for training using their local datasets.
3. Clients use the received model parameters to train their local models for a certain number of epochs. The parameters of the updated local model are then sent back to the server for the aggregation.
4. The server waits till it receives local parameters of all the clients to aggregate them into an updated global model. We applied different aggregation algorithms to aggregate the local model parameters.
5. As we consider client-side evaluation, the server sends the updated global model parameters to all of the clients for evaluation. The clients evaluate the received global model using their local datasets and send the results to the server. The server aggregates the received metrics using a metric aggregation function. In case server-side evaluation is considered, the server evaluates the global model using its local dataset rather than sending the global model to the clients. Even though client-side evaluation adds extra communication overhead, it is more realistic and efficient as the final global model will be sent to the client for prediction.
6. Steps 2–4 are repeated until the desired performance is achieved.

3.2 Aggregation algorithms

To aggregate model updates from different participants, multiple methods were proposed. In order to choose the most effective aggregation method, many algorithms are compared in this study: FedAvg [16], FedProx [24], FedOpt [22], FedAdam [22], FedYogi [22] and FedAdagrad [22].

Table 1 Summary of related work on federated learning-based IDS

References	Attacks studied	Datasets	ML algorithm	Collaborating clients	Convergence	Client-side evaluation	Server-side evaluation
[27]	10	CIC-IdN-IoT [25]	MLP	2,4,8	Not investigated	No evaluation	F1-score
[21]	14	Edge-IIoTset [6]	CNN, RNN	3, 9, 15	Not investigated	accuracy	Accuracy and True Positive Rate
[7]	5,6, 6	TON-IoT [23], Bot-IoT [11], MQTset [29]	CNN, RNN, DNN	5,9,15	Not investigated	Accuracy	Accuracy and True Positive Rate
[13]	7	Gas pipeline system [17]	combined CNN, GRU and MLP	3,5,7	Not investigated	No evaluation	Accuracy, Precision, Recall and F1-score
[3]	4	AWID [9]	Not mentioned	1000	Not investigated	No evaluation	accuracy
[4]	4,6,4	WSN-DS [1], CIC-IDS-2017 [19], KDD CUP 99 [28]	SVN combined with GRU	10	Not investigated	No evaluation	Accuracy, F1-score, False Alarm Rate, Confusion Matrix
[30]	4	CAN messages [12]	NN	9	Not investigated	No evaluation	Accuracy, Precision, Recall and F1-score
[2]	Binary Classification	ToN-IoT	CNN	Not mentioned	Not investigated	No evaluation	Accuracy, Precision, Recall and F1-score
[14]	Binary Classification	custom dataset	GRU	4	Not investigated	No evaluation	Accuracy
[31]	7	CIC-IDS-2017	Temporal Convolutional Network (TCN)	6,10	Not investigated	Accuracy, Precision, Recall and F1-score	Accuracy
[23]	9	CIC-ToN-IoT	Multinomial Logistic Regression (MLR)	10	Not investigated	Accuracy	Accuracy, Precision, Recall and F1-score
[8]	10	Bot-IoT	NN	4	Not investigated	Accuracy, Loss	Accuracy, True Positive Rate, Precision, Recall and F1-score
[18]	5	Custom dataset	GRU	5,9,15	Not investigated	No evaluation	Accuracy, True Positive Rate

3.2.1 FedAvg

FedAvg is a common aggregation method for federated training. In each round, the server sends the global model parameters to a randomly selected group of clients. Each client uses its own local dataset to train the model. Local updates are then sent back to the server for aggregation using Algorithm 1. The updated global model will be sent again to the clients. Local models are trained over multiple epochs which can cut down the number of communication rounds.

Algorithm 1: FedAvg Algorithm

Input: N (number of clients), m (number of participating clients at each round), E (number of local epochs E), η (learning rate), B (batch size B), R (number of rounds)

Server side:

- 1 initialize w_0
- 2 **for** each round t in R **do**
- 3 $G_m \leftarrow$ (random set of m clients).
- 4 **for** each k client in G_m **do**
- 5 sendModel(k, w)
- 6 $w_{t+1}^k \leftarrow \text{ClientTrain}(k, w_t)$
- 7 $w_{t+1} \leftarrow \sum_{k=1}^m \frac{n_k}{n} w_{t+1}^k$

Client side:

- 8 ClientTrain(k, w):
- 9 $\beta \leftarrow$ split P_k into batches of size B
- 10 **for** each epoch e in E **do**
- 11 **for** each batch b in β **do**
- 12 $w \leftarrow w - \eta \nabla l(b, w)$
- 13 return w to server

The algorithm starts by initializing the global model. Then, at the beginning of each round, the algorithm chooses a random set of clients to participate in that round, the global model is then delivered to the participants (lines 3–5). Each client computes local updates $\nabla l(b, w)$ (line 12) through training the model for E epochs, using its local dataset P_k . As soon as the server receives all local parameters, it aggregates them using the formulae in line 7, where n_k represents the size of P_k ($n_k = |P_k|$) and $n = \sum_{k=1}^K n_k$.

3.2.2 FedProx

FedProx is a variant of FedAvg that introduces a regularization term to the FedAvg's local objective function to control the size of local updates. The regularization term is referred to as Proximal term, it helps local updates to stay close to the global model. The local objective function is expressed as 1.

$$h(w, w_t) = F_k(w) + \frac{\nu}{2} \|w - w_t\|^2 \quad (1)$$

$F_k(w)$ denotes the objective function of client K which is the same as in FedAvg. w represents the local parameters to optimize. w_t are the global parameters of round t. The term $\frac{\nu}{2} \|w - w_t\|^2$ is the proximal term. The hyper-parameter ν is used to control the weight of the regularization. With $\nu = 0$, we get the common FedAvg algorithm.

3.2.3 FedOpt, FedAdam, FedAdagrad, FedYogi

Reddi et al. [22] proposed adaptive federated optimization algorithm (FedOpt). The descent optimization method at the client's side is kept as is in the FedAvg. However, the FedAvg aggregating function at the server's side is rewritten as a stochastic gradient descent optimization using a pseudo gradient and a learning rate equals to 1. Three variants of FedOpt were presented using AdaGrad, Adam, and Yogi optimizers. Compared to the FedAvg algorithm, the FedOpt and its variants preserve the same procedures except for the server side's aggregation (line 7) which is rewritten as follows 2:

$$\Delta_t = \sum_{k=1}^m \frac{n_k}{n} \Delta_k^t \quad (2)$$

$$w_{t+1} = \text{ServerOpt}(w_t, -\Delta_t, \eta, t)$$

where $\Delta_k^t = w_k^t - w_t$. w_k^t and w_t are local parameters of client k and global parameters at round t, respectively. *ServerOpt* is the optimization function: SGD in case of FedOpt, AdaGrad in case of FedAdagrad, Adam in case of FedAdam and Yogi in case of FedYogi.

3.3 Client-side evaluation

Inaccurate findings or estimates of model performance may result from relying entirely on server-side evaluation without taking end device performance into account. To guarantee that the model functions properly in real-world scenarios, it is essential to take into account how it behaves on actual client devices. To mitigate this risk, we consider client-side evaluations to evaluate our federated IDS model, for more investigation, we compare it to server-side evaluation.

For server-side evaluation, the server uses its local dataset to evaluate the global model at each round. This evaluation technique assumes that the server has its own local dataset which is not always the case. In client-side evaluation, the global model is evaluated locally using local datasets: At each round, the global model is transferred to the participants, they train it locally and send back local updates to the server for aggregation. The updated global model is sent back again to the participants, but this time, for evaluation. Thus, the participants evaluate the global model and not the local one. Local evaluation metrics (accuracy and loss) are then transferred to the server for aggregation using metric

aggregation function. The FedAvg, Algorithm 2, is updated as follows:

Algorithm 2: FedAvg Algorithm

Input:
 N (number of clients), m (number of participating clients at each round), E (number of local epochs E), η . (learning rate), B (batch size B), R (number of rounds)

Server side:

```

1 initialize  $w_0$ 
2 for each round  $t$  in  $R$  do
3    $G_m \leftarrow$  (random set of  $m$  clients).
4   for each  $k$  client in  $G_m$  do
5     sendModel( $k, w$ )
6      $w_{t+1}^k \leftarrow \text{ClientTrain}(k, w_t)$ 
7    $w_{t+1} \leftarrow \sum_{k=1}^m \frac{n_k}{n} w_{t+1}^k$ 
8   for each  $k$  client in  $G_m$  do
9     sendModel( $k, w_{t+1}$ )
10     $acc_k, loss_k \leftarrow \text{ClientEvaluate}(k, w_{t+1})$ 
11   $aggAcc_t \leftarrow \sum_{k=1}^K \frac{s_k}{s} acc_k$ 
12   $aggLoss_t \leftarrow \sum_{k=1}^K \frac{s_k}{s} loss_k$ 

Client side:
13 ClientTrain( $k, w$ ):
14  $\beta \leftarrow$  split  $P_k$  into batches of size B
15 for each epoch  $e$  in  $E$  do
16   for each batch  $b$  in  $\beta$  do
17      $w \leftarrow w - \eta \nabla l(b, w)$ 
18 return  $w$  to server
19 ClientEvaluate( $k, w$ ):
20  $acc, loss \leftarrow \text{evaluate}(w)$ 
21 return  $acc, loss$  to server
  
```

The results of local evaluations (lines 18 to 20) are aggregated at the server's side using equations in lines 8 to 11. s_k represents the size of the evaluation dataset for client k and $s = \sum_{k=1}^K s_k$.

3.4 Communication efficiency of federated learning

The central server and all involved devices are engaged in multiple rounds of FL training. For each training round, the global model is transferred to all clients for local training using their local datasets. The updated version is transmitted again to the central server for aggregation and then returned back to the end devices. During training rounds, there is a continuous upload and download. Due to limited power and network bandwidth of end devices, the communication overhead of federated training becomes challenging. Thus, our proposed federated-based IDS aims to reduce communication overhead without sacrificing accuracy rate. One way to achieve communication efficiency of training is to reduce the number of rounds. To avoid diminishing accuracy rate, we study the impact of increasing number of epochs per round on

the federated-IDS performance. To this end, we ran a variety of tests, each with a different epoch number.

3.5 Dataset description

The proposed CNN classifier is trained using the CSE-CIC-IDS2018 dataset [26]. The data include the captured network traffic and system logs of each machine in the testbed which is made of a target organization including 420 machines, and 30 servers in addition to the 50 machines that make up the attacking architecture. There are 15 distinct classes in the dataset, as depicted in Table 2. Fourteen are attack varieties, and one is benign.

To reduce the number of input variables to the proposed federated-CNN model, the most consistent and significant features are selected using the random forest Classifier. Out of a total of 80 features, 31 are considered relevant. The synthetic minority oversampling technique (SMOTE) is used to address the issue of minority classes.

3.6 Evaluation metrics

The performance of the proposed IDS is assessed using the following metrics:

- True positive (TP) represents positive samples that were accurately predicted. True negative (TN) represents accurately predicted negative samples. False positive (FP) represents wrongly predicted negative samples. False negative (FN) represents wrongly predicted positive samples.
- Accuracy_{client}: It represents the per-client accuracy. It is the ratio of instances that were accurately predicted, and they are on the main diagonal of the confusion matrix, to all instances. The Accuracy_{client} is calculated as follows¹:

$$\frac{\sum_{j=1}^k TP_j}{\text{Total}} \quad (3)$$

- Precision_{class}: It represents the per-class precision. It is the fraction of TP instances out of all instances predicted as positive. It is calculated as follows:

$$\frac{TP}{TP + FP} \quad (4)$$

- Recall_{class}: It represents the per-class recall. It is the fraction of TP instances out of actual positive instances. It is calculated as follows:

$$\frac{TP}{TP + FN} \quad (5)$$

¹ k represents the number of classes.

Table 2 Attacks in CSE-CIC-IDS2018

1	Bot	2	SSH-Bruteforce	3	DoS-GoldenEye
4	DoS-Slowloris	5	DoS-SlowHTTPTest	6	DDoS attacks-LOIC-HTTP
7	DoS-Hulk	8	DDoS-LOIC-UDP	9	FTP-BruteForce
10	DDoS-HOIC	11	Brute Force -Web	12	Brute Force -XSS
13	SQL Injection	14	Infiltration	15	Benign

- $F1\text{-score}_{\text{class}}$: It represents the per-class F1-score. It is the harmonic mean of $\text{Precision}_{\text{class}}$ and $\text{Recall}_{\text{class}}$. It is calculated as follows:

$$\frac{2 * \text{Precision}_{\text{class}} * \text{Recall}_{\text{class}}}{\text{Precision}_{\text{class}} + \text{Recall}_{\text{class}}} \quad (6)$$

- $\text{Precision}_{\text{client}}$, $\text{Recall}_{\text{client}}$, $F1\text{-score}_{\text{client}}$: They represent, respectively, the per-client precision, recall and F1-score metrics. They are calculated as follows²:

$$\text{Precision}_{\text{client}} = \frac{\sum_{i=1}^k \text{Precision}_{\text{class}}}{k} \quad (7)$$

$$\text{Recall}_{\text{client}} = \frac{\sum_{i=1}^k \text{Recall}_{\text{class}}}{k} \quad (8)$$

$$F1\text{-score}_{\text{client}} = \frac{\sum_{i=1}^k F1\text{-score}_{\text{class}}}{k} \quad (9)$$

- $\text{Accuracy}_{\text{agg}}$: It represents the aggregated accuracy, computed at the aggregator server. It is calculated as follows:

$$\frac{\sum_{\text{client}=1}^k \text{Accuracy}_{\text{client}} * \text{instances}_{\text{client}}}{\text{total of instances}} \quad (10)$$

4 Experimental results

In this section, we evaluate the performance of the proposed federated CNN classifier. Seven clients collaborate in each round to train the CNN model. Thus, the dataset is split into 7 datasets. Class distributions among the clients are depicted in Fig. 1: The attacks are the same for all clients, but they are distributed differently.

The federated classifier is trained using six different aggregation techniques (FedAvg, FedProx, FedOpt, FedAdam, FedYogi and FedAdagrad). For each algorithm, we trained the CNN model for 50 rounds, 2 epochs per round. Figure 2a depicts the server-side accuracy values for the different algorithms. As can be seen FedAvg and FedProx achieved the highest results. The loss metric results are illustrated in Fig. 2b. Both FedAvg and FedProx outperformed the rest of algorithms.

FedAvg and FedProx have similar performance. However, FedProx introduces additional computation overhead due

to local computations of the proximal term at each round. Regarding the limited computational capabilities of edge devices or clients, we opt for FedAvg algorithm to develop our classifier.

4.1 Convergence of the federated-based model

We empirically examine the convergence of the proposed model by running several tests. We set the number of rounds to 50 while altering epoch number for each test. Our choice of altering local computation per round (epochs) is justified by the fact that the epoch number affect the convergence of FedAvg technique [16, 20]. Loss metric is a strong indicator on the model's convergence, thus, we plotted the learning curves in Fig. 3. As can be seen, 2-epoch test models converged from the 20th round, before the other test models. In all below experiments, the federated model is trained for 50 rounds, 2 epochs per round.

4.2 Server-side evaluation

In this section, we analyze the suggested model's performance on the server-side. The server evaluates the newly updated global model using its local dataset after aggregating the local updates. Figure 4 compares performance metrics (loss and accuracy) in case of server-side and client-side evaluations. Server-side metrics are represented by brown bars in both bar-graphs. Clients' accuracies and losses are computed using client-side evaluation. As can be seen, the server-side evaluation does not accurately reflect the federated model's performance: In our scenario, the server-side accuracy rate is the lowest, meanwhile, its loss rate is the highest. Whatever the server-side evaluation results are (higher or lower than the participants), decisions based solely on server-side evaluation are risky because they may include accrediting an IDS model that poorly performs at the end devices (specially when there is significant performance disparity between the server and the end devices). Thus, server-side evaluation is misleading and no longer reliable. It is critical to evaluate client-side performance for an efficient federated-based model.

4.3 Client-side evaluation

Here, we look into the client-side evaluation. As previously stated, after aggregating the various local parameters, the

² k represents the number of classes.

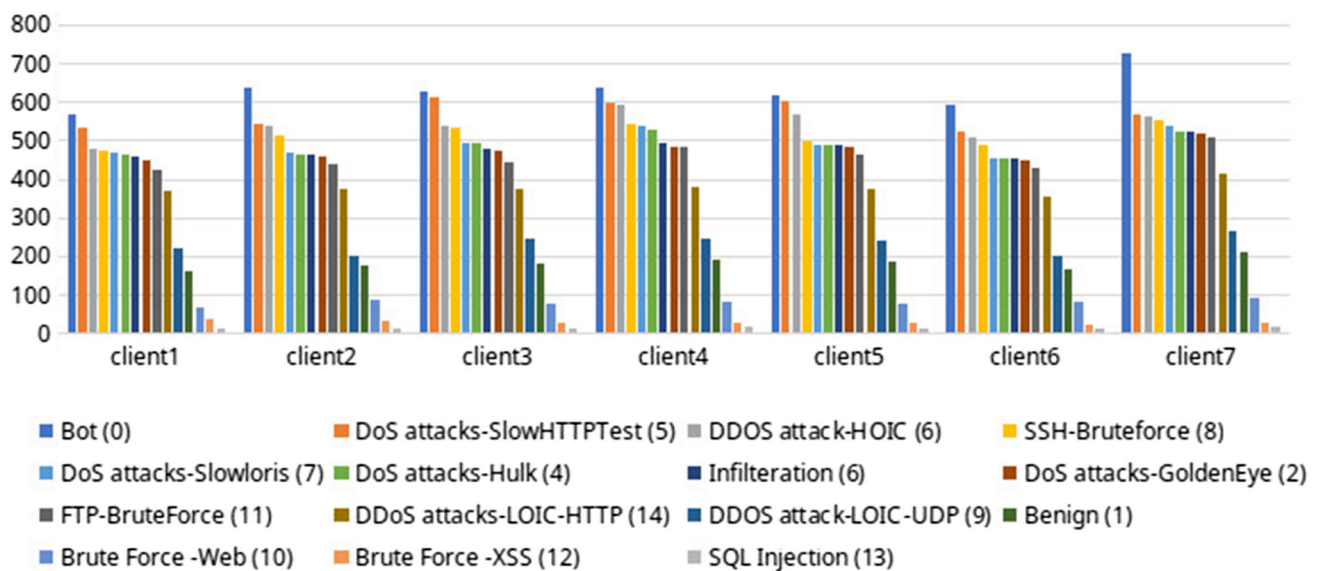


Fig. 1 Class distributions across the collaborating clients

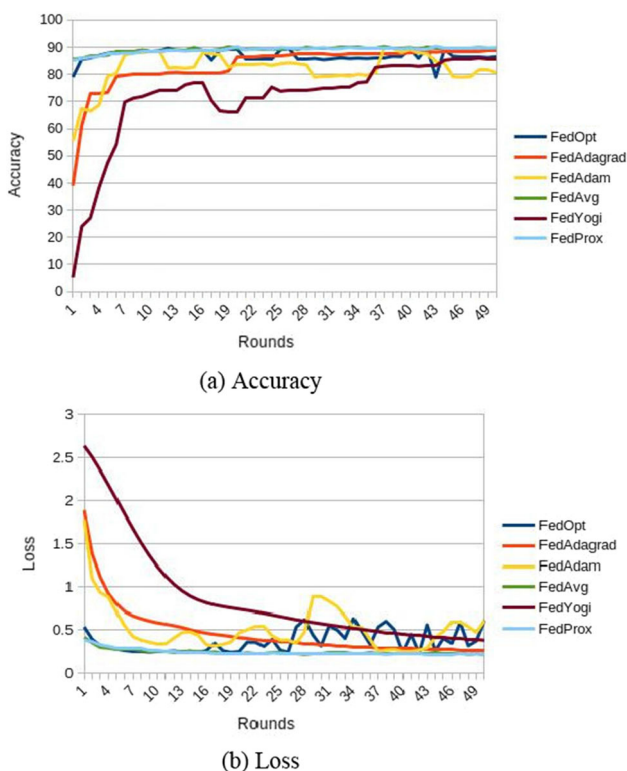


Fig. 2 Comparison of various aggregation algorithms

server delivers the new updated model to the clients for evaluation rather than training. Using its local dataset, each client evaluates the received global model. Results (metric evaluations) are then sent to the server to aggregate them.

The relationships between train accuracy, evaluation accuracy and rounds (respectively, train loss, evaluation-loss and

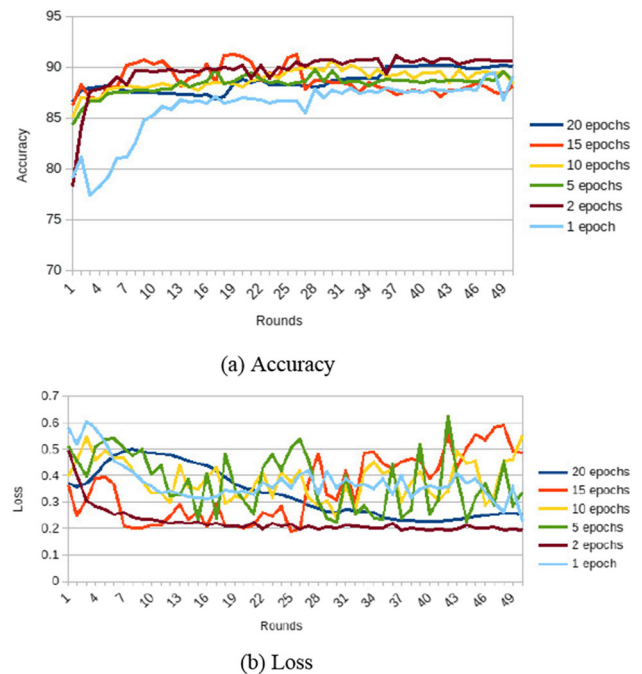


Fig. 3 Comparison of federated performance under different epoch values

rounds) for a random client are shown in Fig. 5. Both accuracy and loss relationships experienced continuous small fluctuations over time. These fluctuations are mostly caused by the fact that the trained/ validated model is a combination of multiple parameters collected from different clients.

Table 3 presents the $Accuracy_{client}$, $Precision_{client}$, $Recall_{client}$, $F1-score_{client}$ metrics for each client. As can be seen, even though the federated clients evaluated the same global model,

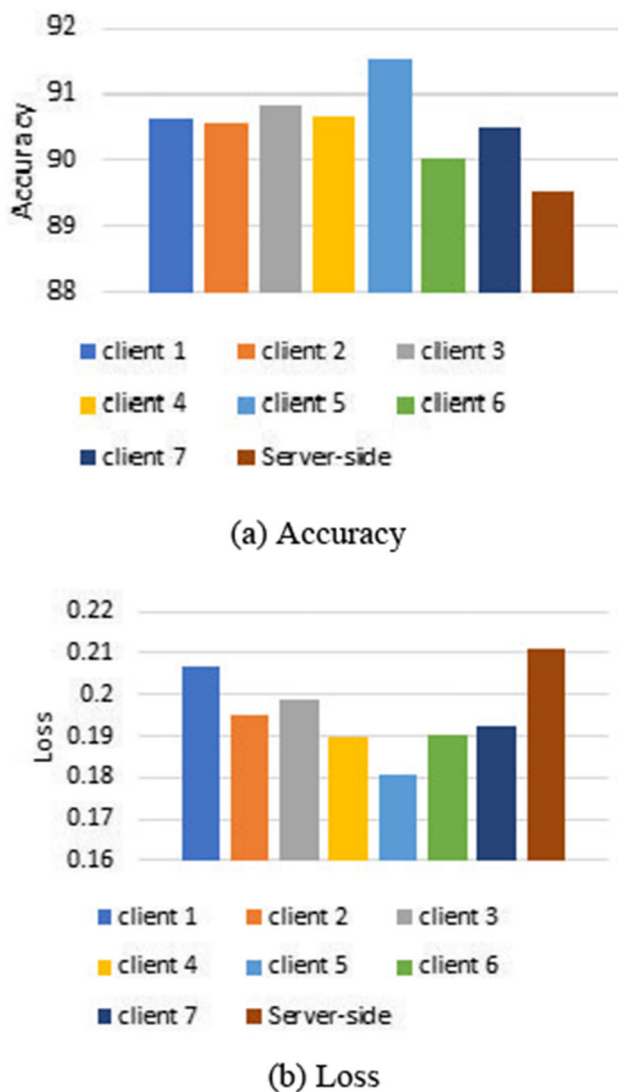


Fig. 4 Server-side versus client-side evaluation

the results differ, which can be attributed to differences in class distributions between local datasets. We believe client-side evaluation can better illustrate the impact of non-IID data on federated learning specially in case of label and feature distribution skew where each client has different distribution.

To further investigate clients' performance, we present in Table 4 the per-client F1-score for each attack³.

We notice that for few attacks, federated clients performed differently. For instance, F1-score decreased by more than 45% for attack 12 in client3 when compared to client7. Surprisingly, client3, when trained individually (Subsect. 4.4), got 1 as F1-score for the same attack. This might be explained by the fact that non-IID data could cause local updates from different clients to clash, resulting in performance degradation of the global model [10].

³ Attack names are matched in Fig. 1

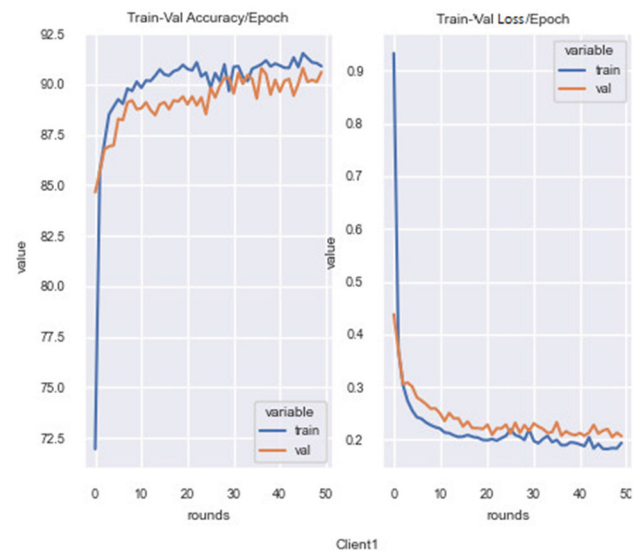


Fig. 5 Federated convergence for a random clients

Table 3 Per-client performance metrics

	Accuracy _{client}	Precision _{client}	Recall _{client}	F1-score _{client}
Client1	91	0.87	0.81	0.8
Client2	91	0.87	0.85	0.83
Client3	91	0.86	0.84	0.81
Client4	91	0.87	0.86	0.84
Client5	92	0.87	0.84	0.84
Client6	90	0.86	0.83	0.82
Client7	90	0.87	0.86	0.84

4.4 Comparison with centralized and individual learning

The proposed federated model is compared to its centralized and individual counterparts:

Centralized Learning: The performance of centralized and federated models for detecting intrusions is compared in terms of accuracy and loss. The centralized CNN model is trained at the server for 50 epochs using the same learning rate and batch size as the federated model. As for the dataset, we used the whole CSE-CIC-IDS2018 dataset without splitting it as in federated learning. The federated model is trained for 50 rounds, 2 epochs per round. We considered client-side evaluation, the aggregated performance metrics are then compared to the centralized model's performance metrics.

Individual learning: Individual learning involves training the same CNN model at each client separately. As we considered 7 federated clients, these clients act as independent devices that train their own model using their

Table 4 $F1\text{--score}_{\text{class}}$ for each client

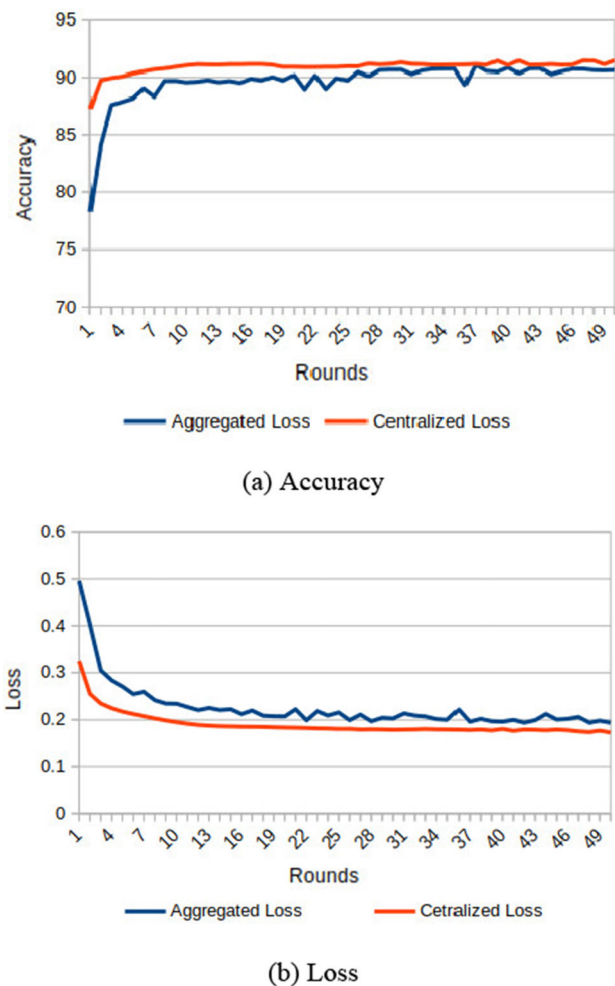
Attack	Client1	Client2	Client3	Client4	Client5	Client6	Client7
0	1	1	1	1	1	1	1
1	0.67	0.53	0.5	0.55	0.52	0.52	0.6
2	0.89	0.87	0.86	0.87	0.88	0.88	0.86
3	1	1	1	1	1	1	1
4	1	1	1	1	1	1	1
5	0.75	0.75	0.75	0.73	0.79	0.73	0.71
6	1	1	1	1	1	1	1
7	1	1	1	1	1	1	1
8	1	1	1	1	1	1	1
9	1	1	1	1	1	1	1
10	0.62	0.76	0.77	0.82	0.74	0.67	0.7
11	0.51	0.52	0.63	0.55	0.61	0.47	0.57
12	0.44	0.72	0.4	0.73	0.83	0.67	0.92
13	0.14	0.26	0.27	0.38	0.27	0.35	0.27
14	1	1	1	1	1	1	1

own local dataset without any cooperation between them. The individual models are trained for 50 epochs using same learning rate, batch size, and local datasets as the federated model.

Centralized and federated performance metrics are plotted in Fig. 6, in which red and blue correspond to centralized and federated metrics, respectively. We observe that centralized model marginally outperformed federated model. The diversity in dataset distributions is thought to be the cause of the performance gap between federated and centralized learning models. Actually, it has been found that the distribution of client data directly affects the effectiveness of FL models. In our case, we have only marginal quantity distribution skew; thus, the difference in performance between federated and centralized learning is marginal. However, in case of highly non-IID datasets, federated learning performs poorly than centralized learning [15].

Each individual's performance is compared to that of its federated equivalent. Thus, each client-side performance is compared to its counterpart individual model. As shown in Fig. 7, federated learning outperforms solo learning. The performance decrease of individual models is attributed to the size of local datasets. This is particularly true for deep learning algorithms as they perform poorly when trained on moderate or small datasets [5]. Data size's impact is getting lessened in federated learning as each client participates in the same learning model (that is, the same model is trained on multiple local datasets).

Figure 8 shows the learning curve for a random client. Compared to Fig. 5 which presents the federated learning curve for the same client, we observe that the individual learning is stable and do not fluctuate. This proves what is

**Fig. 6** Centralized versus federated learning

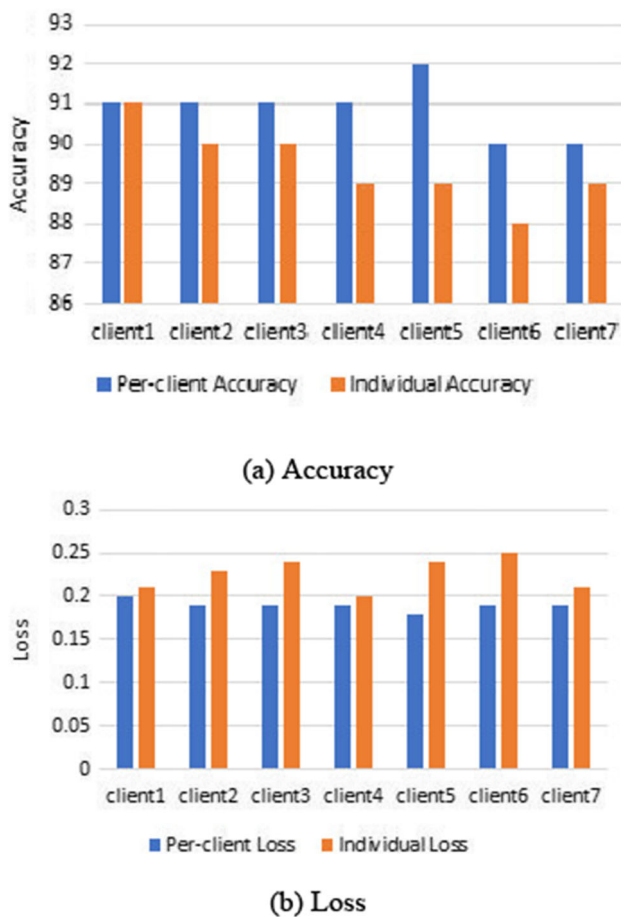


Fig. 7 Individual performance versus federated performance

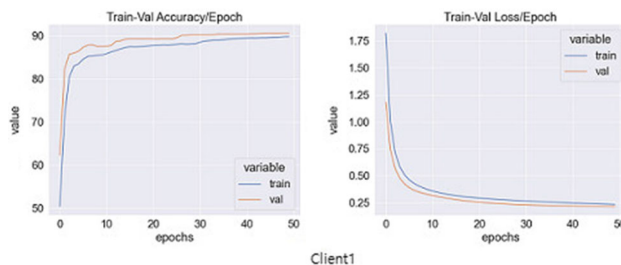


Fig. 8 Learning curve for individual training

observed in Table 4; that is, non-IID data could provoke conflicts between local updates from various clients which lead to performance fluctuations.

5 Future directions

As future work we intend to further examine the resilience of federated-based IDS. In fact, despite the significant liter-

ature on federated-based IDS, their robustness in anomaly detection has not been thoroughly investigated [10, 15].

Although many studies investigated the effectiveness of federated learning under non-IID data settings, all the proposed strategies and investigations considered non-IID data cases among local datasets and not within the same dataset. As each client could have a distinct traffic pattern, certain attacks might be found in the evaluation dataset but absent in the training dataset. Traditional label distribution skew in non-IID datasets is not the issue here as label distributions, in the case of non-IID dataset, vary from one dataset to the next but not within the same dataset. One explanation for such scenarios is the fact that the attack has only recently been discovered and is not yet included in the training dataset but is present in the validation dataset. To achieve a reliable and robust federated-based IDS, it is of utmost importance to investigate its ability to detect unseen attacks. One possible solution to address this issue is to consider developing an a federated-based IDS using real federated environment where each client generates its local dataset rather than splitting one dataset into multiple local datasets. Two scenarios should be investigated and compared: regular IID data scenario and non-IID within the same local dataset scenario. Besides, different aggregating algorithms should be tested. It would be also interesting to investigate non-IID data specially label distribution skew among local datasets in order to provide a deep and full analysis of federated learning with non-IID settings.

6 Conclusion

In this study, we suggested a reliable attack detection and classification method for IoT networks based on federated learning. We trained our model using the CSE-CIC-IDS2018 dataset. Our experiments showed that server-side evaluation could be misleading and do not reflect the true model's performance. Thus, to create a reliable and efficient detection model, the evaluation metrics are computed locally and then sent to the server for aggregation using a metric aggregation function. The model is compared to its centralized and individual counterparts. The experiments results demonstrate that federated and centralized learning have similar performance. However, federated learning has the advantage of protecting data privacy. Compared to individual learning, federated learning achieves higher performance especially in case of small local datasets, as is the case for IoT devices.

Author contributions All the preparation steps were done by NH.

Funding The author certifies that she has no affiliations with or involvement in any organization or entity with any financial interest or

non-financial interest in the subject matter or materials discussed in this manuscript.

Data availability The data that support the findings of this study are available on request from the corresponding author.

Declarations

Conflict of interest The author has no conflict of interest to declare that are relevant to the content of this article.

References

- Almomani, I., Kasasbeh, B., AL-Akhras, M.: WSN-DS: a dataset for intrusion detection systems in wireless sensor networks. *J. Sens.* **2016**, 1–16 (2016). <https://doi.org/10.1155/2016/4731953>
- Arya, M., Sastry, H., Dewangan, B.K., Rahmani, M.K.I., Bhatia, S., Muzaffar, A.W., Bivi, M.A.: Intruder detection in vanet data streams using federated learning for smart city environments. *Electronics*, **12**(4), (2023)
- Cetin, B., Lazar, A., Kim, J., Sim, A., Wu, K.: Federated wireless network intrusion detection. In: 2019 IEEE International Conference on Big Data (Big Data), pp. 6004–6006 (2019)
- Chen, Z., Lv, N., Pengfei Liu, Yu., Fang, K.C., Pan, W.: Intrusion detection for wireless edge networks based on federated learning. *IEEE Access* **8**, 217463–217472 (2020)
- Dawson, H.L., Dubrule, O., John, C.M.: Impact of dataset size and convolutional neural network architecture on transfer learning for carbonate rock classification. *Comput. Geosci.* **171**, 105284 (2023)
- Ferrag, M.A., Friha, O., Hamouda, D., Maglaras, L., Janicke, H.: Edge-IIOTset: A new comprehensive realistic cyber security dataset of IOT and IIOT applications for centralized and federated learning. *IEEE Access* **10**, 40281–40306 (2022)
- Ferrag, M.A., Friha, O., Maglaras, L., Janicke, H., Shu, L.: Federated deep learning for cyber security in the internet of things: concepts applications, and experimental analysis. *IEEE Access* **9**, 138509–138542 (2021)
- Huong, T.T., Bac, T.P., Long, D.M., Thang, B.D., Binh, N.T., Luong, T.D., Phuc, T.K.: Lockedge: Low-complexity cyberattack detection in iot edge computing. *IEEE Access* **9**, 29696–29710 (2021)
- Kolias, C., Kambourakis, G., Stavrou, A., Gritzalis, S.: Intrusion detection in 802.11 networks: empirical evaluation of threats and a public dataset. *IEEE Commun. Surv. Tutor.* **18**(1), 184–208 (2016)
- Kopparapu, K., Lin, E., Zhao, J.: FEDCD: Improving performance in non-IID federated learning. *CoRR*, abs/2006.09637, (2020)
- Koroniotis, N., Moustafa, N., Sitnikova, E., Turnbull, B.: Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset. *Futur. Gener. Comput. Syst.* **100**, 779–796 (2019)
- Lee, H., Jeong, S.H., Kim, H.K.: Otds: A novel intrusion detection system for in-vehicle network by using remote frame. In: 2017 15th Annual Conference on Privacy, Security and Trust (PST), pp. 57–5709, (2017)
- Li, B., Yuhao, W., Song, J., Rongxing, L., Li, T., Zhao, L.: Deepfed: Federated deep learning for intrusion detection in industrial cyber-physical systems. *IEEE Trans. Industr. Inf.* **17**(8), 5615–5624 (2021)
- Li, J., Lyu, L., Liu, X., Zhang, X., Lyu, X.: Fleam: A federated learning empowered architecture to mitigate DDOS in industrial IOT. *IEEE Trans. Industr. Inf.* **18**(6), 4059–4068 (2022)
- Li, Q., Diao, Y., Chen, Q., He, B.: Federated learning on non-IID data silos: An experimental study. *CoRR*, abs/2102.02079, (2021)
- McMahan, H.B., Moore, E., Ramage, D., Agüera y Arcas, B.: Federated learning of deep networks using model averaging. *CoRR*, abs/1602.05629, (2016)
- Morris, T., Gao, W.: Industrial control system traffic data sets for intrusion detection research. In: Jonathan, B., Sujeet, S., (eds), *Critical Infrastructure Protection VIII*, pp. 65–78. Springer Berlin Heidelberg, Berlin, Heidelberg (2014)
- Nguyen, T., Marchal, S., Miettinen, Ma., Fereidooni, H., Asokan, N., Sadeghi, A.-R.: DfIot: A federated self-learning anomaly detection system for IOT, pp. 756–767 (2019)
- Panigrahi, R., Borah, S.: A detailed analysis of cids2017 dataset for designing intrusion detection systems. *Int. J. Eng. Technol.* **7**, 479–482 (2018)
- Qu, Z., Lin, K., Kalagnanam, J., Li, Z., Zhou, J., Zhou, Z.: Federated learning's blessing: Fedavg has linear speedup. *arXiv:2007.05690*, (2020)
- Rashid, M.M., Khan, S.U., Eusufzai, F., Redwan, M.A., Sabuj, S.R., Elsharief, M.: A federated learning-based approach for improving intrusion detection in industrial internet of things networks. *Network* **3**(1), 158–179 (2023)
- Reddi, S.J., Charles, Z., Zaheer, M., Garrett, Z., Rush, K., Konečný, J., Kumar, S., McMahan, H.B.: Adaptive federated optimization. *CoRR*, abs/2003.00295, (2020)
- Rodríguez-Barroso, N., Stipcich, G., Jiménez-López, D., Ruiz-Millán, J.A., Martínez-Cámara, E., González-Seco, G., Luzón, M.V., Veganzones, M.A., Herrera, F.: Federated learning and differential privacy: Software tools analysis, the sherpa.ai fl framework and methodological guidelines for preserving data privacy. *Inf Fusion* **64**, 270–292 (2020)
- Sahu, A.K., Li, T., Sanjabi, M., Zaheer, M., Talwalkar, A., Smith, V.: On the convergence of federated optimization in heterogeneous networks. *CoRR*, abs/1812.06127, (2018)
- Sarhan, M., Layeghy, S., Portmann, M.: Evaluating standard feature sets towards increased generalisability and explainability of ml-based network intrusion detection. *Big Data Res.* **30**, 100359 (2022)
- Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: *International Conference on Information Systems Security and Privacy*, (2018)
- Talpini, J., Sartori, F., Savi, M.: A clustering strategy for enhanced fl-based intrusion detection in IOT networks. *02* (2023)
- Tavallae, M., Bagheri, E., Lu, W., Ghorbani, A.A.: A detailed analysis of the kdd cup 99 data set. In: 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications, pp. 1–6, (2009)
- Vaccari, I., Chiola, G., Aiello, M., Mongelli, M., Cambiaso, E.: Mqttset, a new dataset for machine learning techniques on mqtt. *Sensors* **20**, 11 (2020)
- Yang, J., Hu, J., Yu, T.: Federated ai-enabled in-vehicle network intrusion detection for internet of vehicles. *Electronics* **11**(22), (2022)
- Yang, R., He, H., Yixiao, X., Xin, B., Wang, Y., Yue, Q., Zhang, W.: Efficient intrusion detection toward iot networks using cloud-edge collaboration. *Comput. Netw.* **228**, 109724 (2023)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.